

# Metadata Processing Comparison

Audiobookshelf vs. Bookshelf (Readarr fork)

## 1. Architecture Overview

| Aspect           | Audiobookshelf                                                                                     | Bookshelf                                                                                     |
|------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Language         | JavaScript (Node.js)                                                                               | C# (.NET 6)                                                                                   |
| Provider Pattern | Individual provider classes (Audible, GoogleBooks, OpenLibrary, iTunes, FantLab, Audnexus, Custom) | Single proxy class (BookInfoProxy) that delegates to a remote metadata API (rreading-glasses) |
| Search Entry     | BookFinder.search() — orchestrates all providers                                                   | BookInfoProxy.SearchForNewBook() — delegates to GoodreadsSearchProxy                          |
| Matching         | Client-side fuzzy matching with Levenshtein distance                                               | Server-side matching delegated to metadata API                                                |
| Caching          | None at provider level (stateless)                                                                 | LazyCache + CachedHttpResponseService with 30-60 min TTL                                      |
| Custom Providers | Full custom provider API with OpenAPI spec                                                         | Self-hosted metadata URL only (same API surface)                                              |
| Rate Limiting    | 10s timeout per provider, fail-fast                                                                | Retry loop with Retry-After header, up to 60 retries with 2s sleep                            |

## 2. Key Differences in Matching Strategy

### Audiobookshelf: Multi-Source Fuzzy Matching

Audiobookshelf's BookFinder is the gold standard for this problem space. Key design decisions:

- Multiple providers queried independently — if Google fails, try Audible, OpenLibrary, etc.
- Fuzzy title/author candidates generated by stripping subtitles, bitrate info, edition markers, file types, and 'a novel' suffixes. Each variant is ranked by priority and position.
- Author validation via Audnexus ASIN lookup — candidate authors are cross-checked against a real author database before searching.
- Levenshtein distance filtering with configurable thresholds (default maxTitleDistance=4, maxAuthorDistance=4). Results outside threshold are discarded.
- Match confidence scoring (for Audible) using weighted formula: 70% duration match + 20% title similarity + 10% author similarity. Duration is the strongest signal because audiobook runtimes are unique fingerprints.
- Up to 5 fuzzy search retries with progressively cleaned title/author combinations before giving up.
- ISBN/ASIN direct lookup paths bypass fuzzy matching entirely when identifiers are available.

### Bookshelf: Centralized Metadata API Proxy

Bookshelf takes a fundamentally different approach — it outsources matching to a remote metadata API:

- Single metadata source — all queries go through rreading-glasses (GoodReads scraper or Hardcover API). No fallback providers.
- No client-side fuzzy matching — the search query is sent verbatim to GoodreadsSearchProxy, which returns IDs. No Levenshtein distance, no title cleaning, no candidate generation.
- Search by identifier works well — supports author:ID, work:ID, edition:ID, isbn:VALUE, asin:VALUE prefix syntax.
- Edition selection is popularity-based — monitors the most popular edition by default (OrderByDescending rating \* count).

- Heavy retry logic for rate limiting (up to 60 iterations with 2s sleep), but no fallback if the API itself returns poor results.
- No confidence scoring — results are returned as-is from the metadata API with no quality ranking.

## 3. What Bookshelf Should Adopt from Audiobookshelf

### 3.1 Client-Side Fuzzy Matching (High Priority)

This is the single biggest improvement opportunity. Many of Bookshelf's open issues (#59, #92, #109, #131) trace back to the lack of fuzzy matching. When the metadata API returns slightly different title/author formatting than what the user searched for, Bookshelf fails silently.

- Implement Levenshtein distance or similar string similarity scoring in the search pipeline.
- Add title normalization — strip subtitles, parenthetical content, edition markers, accented characters.
- Add author normalization — handle middle initials, 'et al.', accented characters, first-last / last-first variations.
- Generate multiple search candidates from a single query, ranked by likelihood.

### 3.2 Multi-Provider Support (Medium Priority)

Audiobookshelf queries Google Books, Open Library, Audible, iTunes, FantLab, and custom providers. Bookshelf has exactly one. Adding even Google Books as a fallback would dramatically improve hit rates for books that GoodReads/Hardcover metadata handles poorly.

- Add a provider interface (IMetadataProvider) with Search, GetByISBN, GetByASIN methods.
- Implement Google Books and Open Library as additional providers.
- Add provider priority/fallback logic in the search pipeline.

### 3.3 Match Confidence Scoring (Medium Priority)

Audiobookshelf's confidence formula (duration + title similarity + author similarity) is elegant and effective. For ebooks, duration isn't available, but page count could serve a similar role.

- Score results by: title similarity (40%) + author similarity (30%) + page count proximity (20%) + ISBN/ASIN exact match bonus (10%).
- Surface confidence scores in the UI so users can see why a match was chosen.
- Allow users to set minimum confidence thresholds.

### 3.4 Custom Provider API (Lower Priority)

Audiobookshelf has a full OpenAPI spec for custom metadata providers. Bookshelf only supports changing the metadata URL. A proper custom provider API would let the community build integrations without forking.

## 4. What Bookshelf Does Well

- Identifier-based search (author:ID, work:ID, isbn:VALUE) is clean and well-implemented.
- Edition tracking and monitoring is more sophisticated than audiobookshelf — tracks multiple editions per work, monitors the most popular, and supports user overrides.
- Series linking is comprehensive — full series/work relationship mapping with position tracking.
- Rate limit handling is robust — retries with Retry-After header respect.
- Dual metadata backend (GoodReads softcover / Hardcover hardcover) gives users a choice that audiobookshelf doesn't offer.

## 5. Implementation Priority

| Priority | Change | Effort | Impact |
|----------|--------|--------|--------|
|----------|--------|--------|--------|

|    |                                                                 |                   |                                       |
|----|-----------------------------------------------------------------|-------------------|---------------------------------------|
| P0 | Add Levenshtein distance + title/author normalization to search | Medium (2-3 days) | Fixes #59, #92, #109, #131            |
| P0 | Generate fuzzy search candidates from query                     | Medium (2-3 days) | Dramatically improves search hit rate |
| P1 | Add Google Books as fallback provider                           | Medium (3-5 days) | Covers GoodReads/Hardcover gaps       |
| P1 | Add Open Library as fallback provider                           | Low (1-2 days)    | Simple API, good ISBN coverage        |
| P1 | Implement match confidence scoring                              | Low (1-2 days)    | Better result ranking                 |
| P2 | Add custom provider API spec                                    | Medium (3-5 days) | Community extensibility               |
| P2 | Add Audible/Audnexus for audiobook metadata                     | Medium (3-5 days) | Better audiobook support              |